

Revisit to CPE233 (Digital Design)

Authors:
WhisperingRock

Professor:

Class
Quarter
July 19, 2026

Contents

- 0.1 Hardware 2
 - 0.1.1 HW1 - Program ROM 2
 - 0.1.2 HW2 - Program Counter 5
- 0.2 Software 11
 - 0.2.1 SW1 - Intro to Assembly Lang 11
 - 0.2.2 SW2 - Conditional Statements 20
- 0.3 Appendix 24

Note : All assignments are in the CPE 233 Lab Manual.

0.1 Hardware

0.1.1 HW1 - Program ROM

Reverse Engineer’d Table

0x0000_0000:	11000537
0x0000_0004:	00a00f13
0x0000_0008:	00051783
0x0000_000C:	41e7da33
0x0000_0010:	00fa4633
0x0000_0014:	04c52023
0x0000_0018:	ff1ff06f

Figure 1: Which Table? This table

RISC-V Instruction Set Summary

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op
imm11,0		rs1	funct3	rd	op
imm11,5	rs2	rs1	funct3	imm4,0	op
imm12,10,5	rs2	rs1	funct3	imm4,1,11	op
imm31,12				rd	op
imm20,10,1,11,19,12				rd	op
fs3	funct2	fs2	fs1	funct3	fd
5 bits	2 bits	5 bits	3 bits	5 bits	7 bits

R-Type

I-Type

S-Type

B-Type

U-Type

J-Type

R4-Type

- imm: signed immediate in imm_{11,0}
- uimm: 5-bit unsigned immediate in imm_{4,0}
- upimm: 20 upper bits of a 32-bit immediate, in imm_{31,12}
- Address: memory address: rs1 + SignExt(imm_{11,0})
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt((imm_{12,1,1'b0}))
- JTA: jump target address: PC + SignExt((imm_{20,1,1'b0}))
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

Figure B.1 RISC-V 32-bit instruction formats

Table B.1 RV32I: RISC-V integer instructions

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address]7:0)
0000011 (3)	001	–	I	lh rd, imm(rs1)	load half	rd = SignExt([Address]15:0)
0000011 (3)	010	–	I	lw rd, imm(rs1)	load word	rd = [Address]31:0
0000011 (3)	100	–	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address]7:0)
0000011 (3)	101	–	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address]15:0)
0010011 (19)	000	–	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	–	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	–	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	–	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srlr rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srair rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	–	I	orir rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	–	I	andir rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	–	–	U	auipc rd, upimm	add upper immediate to PC	rd = (upimm, 12'b0) + PC
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	[Address]7:0 = rs27:0
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	[Address]15:0 = rs215:0
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	[Address]31:0 = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 – rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs24:0
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs24:0
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs24:0
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	–	–	U	lui rd, upimm	load upper immediate	rd = (upimm, 12'b0)
1100011 (99)	000	–	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	–	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	–	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	–	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	–	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	–	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	–	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	–	–	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

*Encoded in instr1,259 the upper seven bits of the immediate field

ProgROM Addr	Machine Code	Type	Assembly Instruction
0000	0x1100 0537		
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui rd, upimm
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui x10, 0x11000
0004	0x00A0 0F13		
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi rd, rs1, imm
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi x30, x0, 0x00A
0008	0x0005 1783		
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh rd, imm(rs1)
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh x15, 0(x10)
000C	0x41E7 DA33		
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra rd, rs1, rs2
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra x20, x15, x30
0010	0x00FA 4633		
	0000 0000 1111 1010 0 100 0110 0 011 0011	(51)R	xor rd, rs1, rs2
	0000 000 0 1111 1010 0 100 0110 0 011 0011	(51)R	xor x12, x20, x15
0014	0x04C5 2023		
	0000 0100 1100 0101 0 010 0000 0 010 0011	(35)S	sw rs2, imm(rs1)
	0000 010 0 1100 0101 0 010 0000 0 010 0011	(35)S	sw x12, 0x040(x10)
0018	0xFF1F F06F		
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal rd, label
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal x0, -16

Table 1: Reverse Engineering *otter_memory.mem* segment

For the last command, some explanation is required here. Consider

- $Imm[20] = B31 = 1$
- $Imm[10 : 1] = B30 : B21 = 1111111000$
- $Imm[11] = B20 = 1$
- $Imm[19 : 12] = B19 : B12 = 11111111$
- $Imm[0]$ is always zero
- Therefore, $Imm = 0x1FFFF0$

To find the decimal representation, consider adding our large negative number to the next highest max.

$$\begin{aligned}
 2^{21} - 0x1FFFF0 &= 0x200000 - 0x1FFFF0 \\
 &= -0x10 \\
 &= -16
 \end{aligned}$$

BUT WAIT! jal doesn't take immediates, only labels.

True, our label must be 4 spaces prior to this jump - fill this portion in with a label in the asm and respect the spacing.

Assembly Code

Listing 1: Assembly of HW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Confirm decoded machine code (by hand)
4 #           functionality matches what the RARS compiler
5 #           produces.
6
7
8
9 .data
10 A: .word 0x1234    # dummy value to see change
11
12 .global _start
13
14 .text
15 .equ MMIO, 0x11000000
16
17 _start:
18 # ~~~~ init registers ~~~~
19 li t1, MMIO
20 lw t2, A
21 sw t2, 0(t1)
22
23 # ~~~~ decoded instr ~~~~

```

```
24  #lui x10, 0x11000
25  #addi x30, x0, 0x00A
26  #loop:
27  #lh x15, 0(x10)
28  #sra x20, x15, x30
29  #xor x12, x20, x15
30  #sw x12, 0x040(x10)
31  #jal x0, loop    #loop label is 4 spaces back
32
33  lui a0, 0x11000    # a0 = x10
34  addi t5, zero, 0x00A    # zero = x0, t5 = x30
35  loop:
36  lh a5, 0(a0)        # a5 = x15
37  sra s4, a5, t5        # s4 = x20
38  xor a2, s4, a5        # a2 = x12
39  sw a2, 64(a0)
40  jal zero, loop
41
42  # ~~~~ syscall exit status ~~~~
43  li a7, 10    # load syscall call num for exit()
44  li a0, 0      # EXIT_SUCCESS code
45  ecall        # make the system call
```

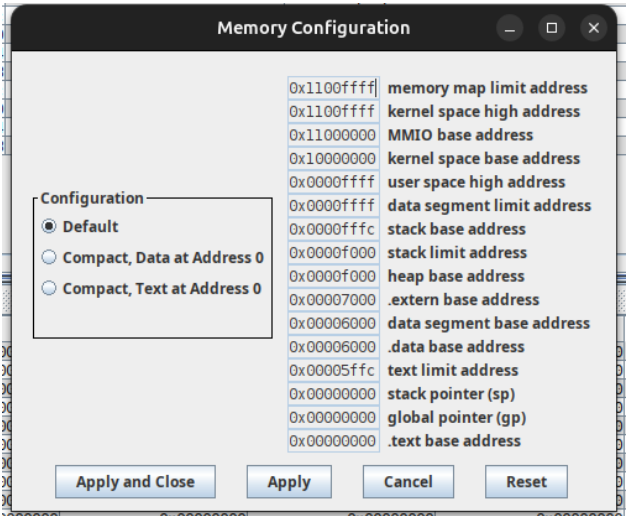


Figure 2: Segment Address Config

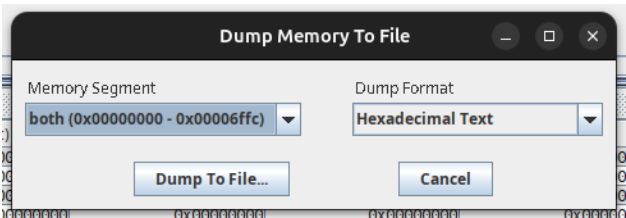


Figure 3: Memory Dump Config

Text Segment				
Bkpt	Address	Code	Basic	Source
	0x00000000	0x11000337	lui x6,0x00011000	19: li t1, 0x11000000
	0x00000004	0x00030313	addi x6,x6,0	
	0x00000008	0x00006397	auipc x7,6	20: lw t2, A
	0x0000000c	0xff83a383	lw x7,0xfffffff8(x7)	
	0x00000010	0x00732023	sw x7,0(x6)	21: sw t2, 0(t1)
	0x00000014	0x11000537	lui x10,0x00011000	33: lui a0, 0x11000 # a0 = x10
	0x00000018	0x00a00f13	addi x30,x0,10	34: addi t5, zero, 0x00A # zero = x0, t5 = x30
	0x0000001c	0x00051783	lh x15,0(x10)	36: lh a5, 0(a0) # a5 = x15
	0x00000020	0x41e7da33	sra x20,x15,x30	37: sra s4, a5, t5 # s4 = x20
	0x00000024	0x00fa4633	xor x12,x20,x15	38: xor a2, s4, a5 # a2 = x12
	0x00000028	0x04c52023	sw x12,0x00000040(x10)	39: sw a2, 64(a0)
	0x0000002c	0xff1ff06f	jal x0,0xfffffff0	40: jal zero, loop
	0x00000030	0x00a00893	addi x17,x0,10	43: li a7, 10 # load syscall call num for exit()
	0x00000034	0x00000513	addi x10,x0,0	44: li a0, 0 # EXIT_SUCCESS code
	0x00000038	0x00000073	ecall	45: ecall # make the system call

Figure 4: RARS instruction addresses alongside code

Vivado Simulation

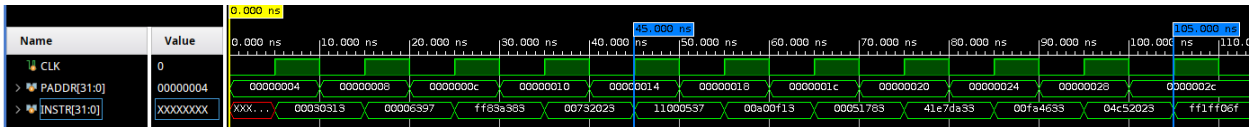


Figure 5: Simulation waveform of ProgROM_tb.sv running otter_memory_hw1.mem

By inspection, the instructions in the waveform 5 match those in RARS4 and our decoded table 1

0.1.2 HW2 - Program Counter

Intended Behavior(s)

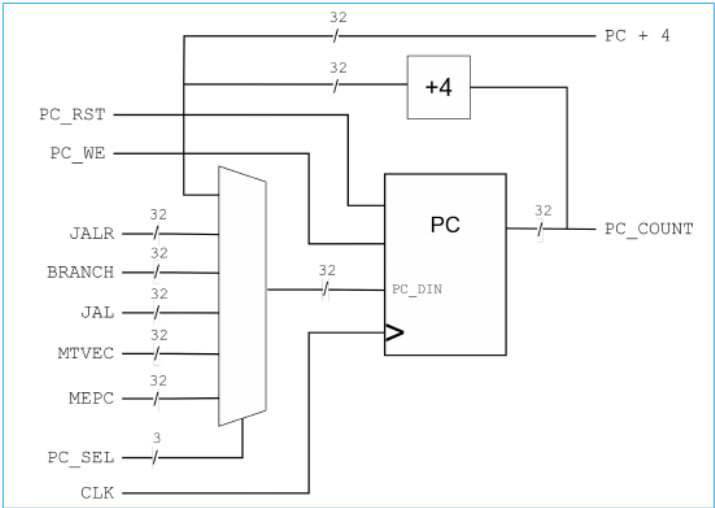


Figure 6: Program Counter Test Block Design

Structural Design(s)

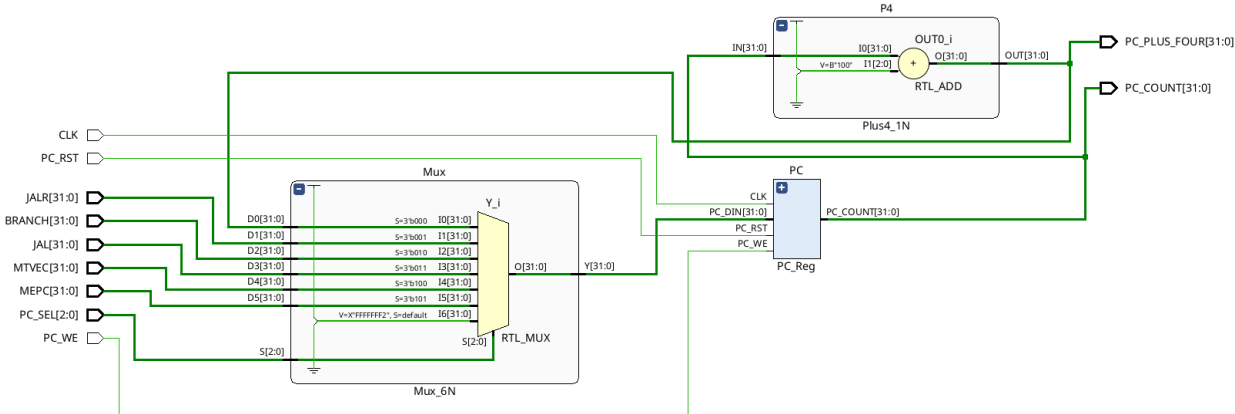


Figure 7: RTL Design of PC_TestBlock.sv

Synthesis Warning(s)

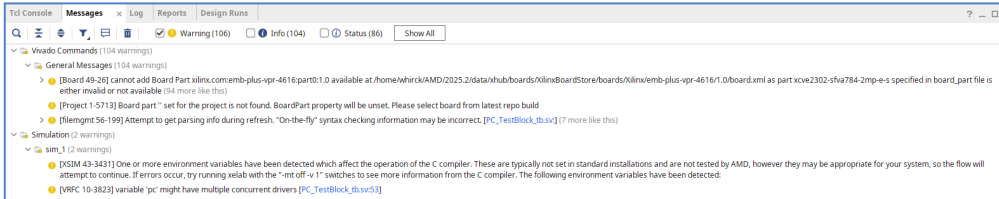


Figure 8: Synthesis Errors of PC_TestBlock.sv

The only error with importance here is 'clk' having multiple concurrent drivers such that one clock toggles clk and the other initially defines the clk value. I opted to proceed with this error.

Verification

All test cases passed in PC_TestBlock_tb.sv passed in simulation along with each subcomponent test bench.

Listing 2: Simulation file for PC_TestBlock.sv

```
1 'timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 07/19/2026 09:39:05 AM
7 // Design Name:
8 // Module Name: PC_TestBlock_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
```

```

13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic reset, write, clk;
27     logic[2:0] sel;
28     logic[5:0][31:0] d;           // d[0] is unused b/c its p+4 on mux
29     logic[31:0] pc, pc_plus_4;
30
31     // ~~~~ instances ~~~~
32     PC_TestBlock UUT(
33         .PC_RST(reset),
34         .PC_WE(write),
35         .CLK(clk),
36         .PC_SEL(sel),
37         .JALR(d[1]),
38         .BRANCH(d[2]),
39         .JAL(d[3]),
40         .MTVEC(d[4]),
41         .MEPC(d[5]),
42         .PC_COUNT(pc),
43         .PC_PLUS_FOUR(pc_plus_4)
44     );
45
46     // ~~ action : CLK (10ns period) ~~
47     always begin
48         #5;
49         clk <= !clk;
50     end
51
52     // ~~~~ action : data ~~~~
53     initial begin
54
55         // ~~ init ~~
56         clk = 1; reset = 0; write = 0; sel = 0;
57         d[1] = 32'h1234_5678;
58         d[2] = 32'hBAAC_4444;
59         d[3] = 32'h0000_01A1;
60         d[4] = 32'hBEEF_BEEF;
61         d[5] = 32'hDEAD_DEAD;
62         pc = 0; pc_plus_4 = 0;
63         #10;
64
65         // ~~ TC1 : sel all w/ write disabled ~~
66         for (int i = 0; i < 16; i++) begin
67             sel = i;
68             #10;
69             assert(pc === 0) else $error("TC1: sel failed (pc)");
70             assert(pc_plus_4 === 4) else $error("TC1: sel failed (pc+4)");
71         end
72
73
74
75         // ~~ TC2 : sel all w/ write enabled ~~
76         reset = 1; sel = 0; #10; reset = 0; #10;
77         write = 1; #10;
78
79         assert(pc === 4) else $error("TC2: sel(0) failed on pc");           // Not zero b/c d[0] is 0+4 already
80         assert(pc_plus_4 === 8) else $error("TC2: sel(0) failed on pc+4");
81
82         for(int i=1; i < 6; i++) begin
83             sel = i; #10;
84             assert(pc === d[i]) else $error("TC2: failed on pc");
85             assert(pc_plus_4 === (d[i]+4) ) else $error("TC2: failed on pc+4");
86         end
87
88
89         // ~~ TC3 : inc pc ~~
90         reset = 1; sel = 0; #10; reset = 0;
91         for(int i=1; i < 256; i++) begin
92             #10;
93             assert(pc === (4*i)) else $error("TC3: failed on pc");
94             assert(pc_plus_4 === (4+(4*i)) ) else $error("TC3: failed on pc+4");
95         end
96
97
98         // ~ test conclusion ~
99         reset = 1;
100     end
101 endmodule

```

Listing 3: Simulation file for Mux_6N.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 04:26:27 PM
7  // Design Name:
8  // Module Name: Mux_6N_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Mux_6N_tb();
24
25     // ~~ local variables ~~
26     logic CLK;
27     logic[31:0] d0, d1, d2, d3, d4, d5;
28     logic[3:0] sel;
29     logic[31:0] out;
30
31
32     // ~~ high-level module instances (and hooks) ~~
33     Mux_6N #(32) UUT (
34         .D0(d0),
35         .D1(d1),
36         .D2(d2),
37         .D3(d3),
38         .D4(d4),
39         .D5(d5),
40         .S(sel),
41         .Y(out)
42     );
43
44     // ~~ action : CLK (10ns period) ~~
45     always begin
46         #5;
47         CLK <= !CLK;
48     end
49
50     // ~~ action : Data ~~
51     initial begin
52
53         // ~ init ~
54         CLK = 0;
55         d0 = 32'h0000_0000;
56         d1 = 32'h1234_0000;
57         d2 = 32'h0000_5678;
58         d3 = 32'h1111_1111;
59         d4 = 32'hFFFF_FFFF;
60         d5 = 32'hDEAD_BEEF;
61
62         // ~ selector test ~
63         sel = 0; #10
64         assert(out == d0) else $error("sel = 0 failed");
65
66         sel = 1; #10
67         assert(out == d1) else $error("sel = 1 failed");
68
69         sel = 2; #10
70         assert(out == d2) else $error("sel = 2 failed");
71
72         sel = 3; #10
73         assert(out == d3) else $error("sel = 3 failed");
74
75         sel = 4; #10
76         assert(out == d4) else $error("sel = 4 failed");
77
78         sel = 5; #10
79         assert(out == d5) else $error("sel = 5 failed");
80
81         sel = 0; #10
82         assert(out == d0) else $error("sel = 0 failed");
83
84         sel = 2; #10
85         assert(out == d2) else $error("sel = 2 failed");
86
87         sel = 0; #10
88         assert(out == d0) else $error("sel = 0 failed");
89

```

```

90     sel = 4; #10
91     assert(out === d4) else $error("sel = 4 failed");
92
93     // ~ edge case : input changes ~
94     sel = 3; #10
95     assert(out === d3) else $error("sel = 3 failed");
96
97     d3 = 32'hF000_0000; #10
98     assert(out === 32'hF000_0000) else $error("h7 failed");
99
100    d3 = 32'h0F00_0000; #10
101    assert(out === 32'h0F00_0000) else $error("h6 failed");
102
103    d3 = 32'h00F0_0000; #10
104    assert(out === 32'h00F0_0000) else $error("h5 failed");
105
106    d3 = 32'h000F_0000; #10
107    assert(out === 32'h000F_0000) else $error("h4 failed");
108
109    d3 = 32'h0000_F000; #10
110    assert(out === 32'h0000_F000) else $error("h3 failed");
111
112    d3 = 32'h0000_0F00; #10
113    assert(out === 32'h0000_0F00) else $error("h2 failed");
114
115    d3 = 32'h0000_00F0; #10
116    assert(out === 32'h0000_00F0) else $error("h1 failed");
117
118    d3 = 32'h0000_000F; #10
119    assert(out === 32'h0000_000F) else $error("h0 failed");
120
121    // ~ default case ~
122    sel = 6;
123    while(sel < 15) begin
124        #10;
125        assert(out === (32'hFFFF_FFFF - 13)) else $error("default %d failed", sel);
126        sel = sel + 1;
127    end
128
129 end
130
131
132 endmodule

```

Listing 4: Simulation file for PC_Reg.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 06:36:03 PM
7  // Design Name:
8  // Module Name: PC_Reg_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_Reg_tb();
24
25     // ~~ local variables ~~
26     logic clk, reset, write;
27     logic[31:0] din, dout;
28     logic[31:0] answer;
29
30
31     // ~~ high-level module instances (and hooks) ~~
32     PC_Reg UUT (
33         .CLK(clk),
34         .PC_RST(reset),
35         .PC_WE(write),
36         .PC_DIN(din),
37         .PC_COUNT(dout)
38     );
39
40     // ~~ action : CLK (10ns period) ~~
41     always begin
42         #5;
43         clk <= !clk;
44     end
45

```

```

46 // ~~ action : Data ~~
47 initial begin
48
49     // ~ init ~
50     clk = 1; reset = 0; write = 0;
51     reset = 1; #10; // zero out register
52
53     // ~ test cases ~
54     answer = 32'hDEAD_BEEF;
55     din = answer;
56
57     // ~ TC1 : write ~
58     reset = 0; #10;
59     assert(dout == 0) else $error("TC1: init failed");
60     write = 1; #10;
61     assert(dout == answer) else $error("TC1: write failed");
62
63     // ~ TC2: reset after write ~
64     reset = 1; #10;
65     assert(dout == 0) else $error("TC2: reset failed");
66
67     // ~ TC3: reset priority over write ~
68     write = 1; reset = 1; #10;
69     assert(dout == 0) else $error("TC3: reset failed");
70
71
72     // ~ TC4 : write within range ~
73     write = 1; reset = 0; #10;
74     answer = 32'h0246_8ACE;
75     for (int i = 0; i < 8; i++) begin
76         din = answer; #10;
77         assert(dout == answer) else $error("TC4: write failed");
78         answer = answer << 4;
79     end
80
81     // ~ TC5 : Rollover ~
82     din = 32'hFFFF_FFFF; #10;
83     assert(dout == din) else $error("TC5: write failed");
84     din = din + 1; #10;
85     assert(dout == 0) else $error("TC5: rollover failed");
86
87
88
89
90 end
91
92
93 endmodule

```

SystemVerilog Code

Listing 5: PC_TestBlock.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 07:34:25 PM
7  // Design Name:
8  // Module Name: PC_TestBlock
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock(
24     input logic PC_RST, PC_WE, CLK,
25     input logic[2:0] PC_SEL,
26     input logic[31:0] JALR, BRANCH, JAL, MTVEC, MEPC,
27     output logic[31:0] PC_COUNT, PC_PLUS_FOUR
28 );
29
30 // ~~~~ inits and wires ~~~~
31 logic[31:0] MUX_0_WIRE;
32
33 // ~~~~ instances ~~~~
34 Mux_6N #(32) Mux(
35     .D0(PC_PLUS_FOUR),
36     .D1(JALR),

```

```

37         .D2(BRANCH),
38         .D3(JAL),
39         .D4(MTVEC),
40         .D5(MEPC),
41         .S(PC_SEL),
42         .Y(MUX_O_WIRE)
43     );
44
45
46     PC_Reg PC(
47         .CLK(CLK),
48         .PC_RST(PC_RST),
49         .PC_WE(PC_WE),
50         .PC_DIN(MUX_O_WIRE),
51         .PC_COUNT(PC_COUNT)
52     );
53
54     Plus4_1N #(32) P4(
55         .IN(PC_COUNT),
56         .OUT(PC_PLUS_FOUR)
57     );
58
59
60 endmodule

```

0.2 Software

0.2.1 SW1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

Flowchart

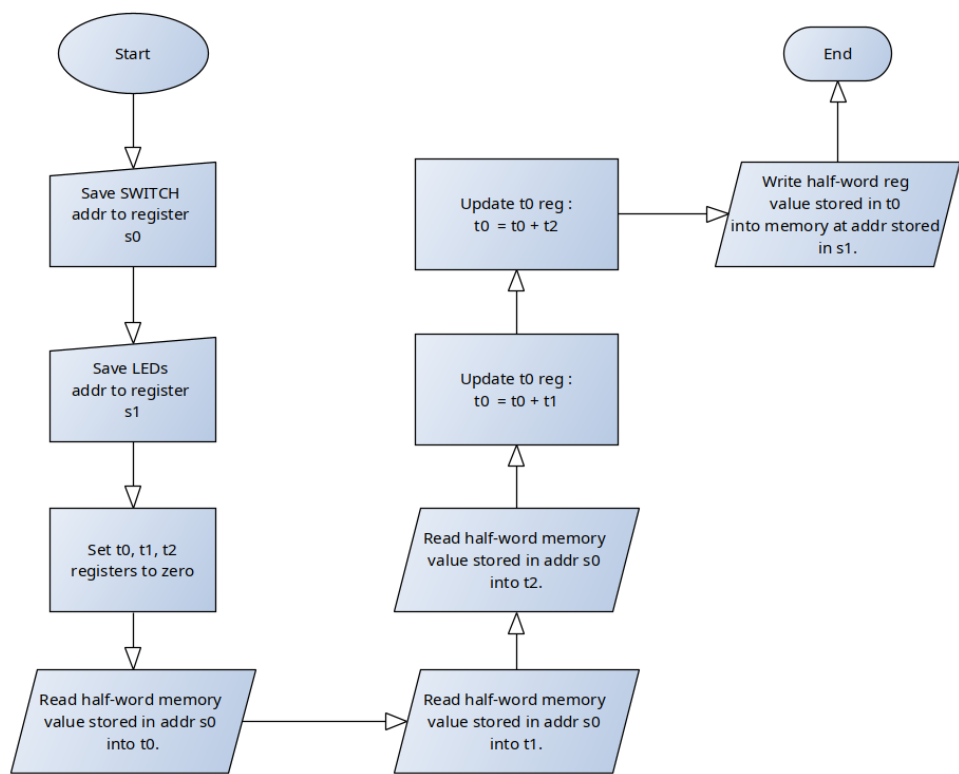


Figure 9: Flow Chart for Part 1

Verification Simulations

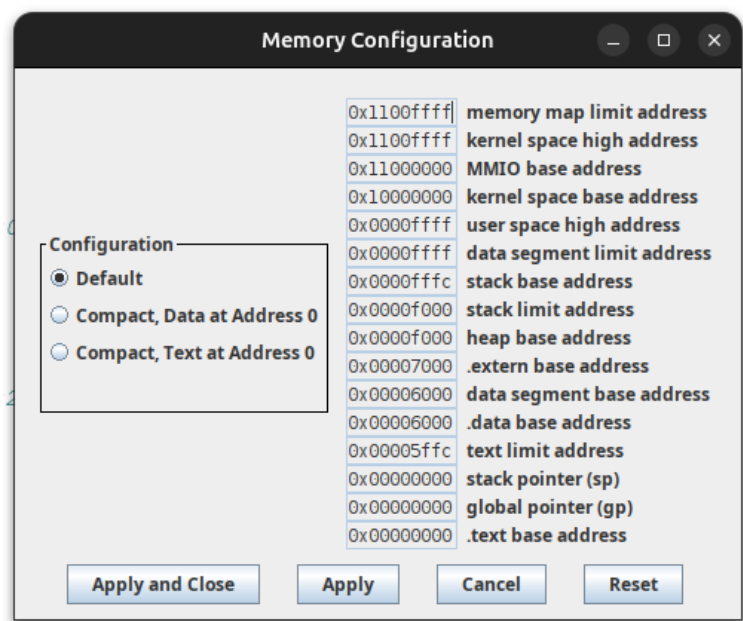


Figure 10: RARS Memory Configuration

```

20
21 #.equ SWITCHES_H,      0x11000 # upper 20 bits
22 #.equ SWITCHES_L,      0x000    # lower 12 bits
23 .equ SWITCHES_H,       0x00006 # upper 20 bits (data seg at 0x00006000)
24 .equ SWITCHES_L,       0x000    # lower 12 bits
25
26 #.equ LEDS_H,          0x11000 # upper 20 bits
27 #.equ LEDS_L,          0x020    # lower 12 bits
28 .equ LEDS_H,           0x00006 # upper 20 bits (data seg at 0x00006020)
29 .equ LEDS_L,           0x020    # lower 12 bits
30

```

Figure 11: Address Correction to work with RARS

- Test Case 1

```

33 # TC1 : Word Clip
34 .equ READ1_H,          0x00001 # switch first read (0x1234)
35 .equ READ1_L,          0x234
36
37
38 .equ READ2_H,          0x12345 # switch second read (0x5678)
39 .equ READ2_L,          0x678
40
41 .equ READ3_H,          0x11111 # switch third read (0x1111)
42 .equ READ3_L,          0x111
43 #   answer= 0x79BD
44

```

Figure 12: TC1 given inputs

The screenshot shows the RARS IDE interface. The 'Text Segment' table lists memory addresses from 0x00000000 to 0x0000004C with their corresponding assembly instructions. The 'Data Segment' table shows memory addresses 0x00006000 and 0x00006020, both containing the value 0x00000000.

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	46: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x00000313	addi x6,x0,0	47: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x00000393	addi x7,x0,0	48: addi t2, zero, 0 # t2 is the SW third read
	0x0000000C	0x00006437	lui x8,6	56: lui s0, 0x00006
	0x00000010	0x00040413	addi x8,x8,0	57: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	59: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	60: addi s1, s1, 0x020
	0x0000001C	0x00001e37	lui x28,1	66: lui t3, 0x00001
	0x00000020	0x234e0e13	addi x28,x28,0x0000234	67: addi t3, t3, 0x234
	0x00000024	0x01c42023	sw x28,0(x8)	68: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	71: lhu t0, 0(s0) # first read
	0x0000002C	0x12345e37	lui x28,0x00012345	74: lui t3, 0x12345
	0x00000030	0x678e0e13	addi x28,x28,0x00000678	75: addi t3, t3, 0x678
	0x00000034	0x01c42023	sw x28,0(x8)	76: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	79: lhu t1, 0(s0) # second read
	0x0000003C	0x11111e37	lui x28,0x00011111	82: lui t3, 0x11111
	0x00000040	0x111e0e13	addi x28,x28,0x00000111	83: addi t3, t3, 0x111
	0x00000044	0x01c42023	sw x28,0x00000111	84: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	87: lhu t2, 0(s0) # third read
	0x0000004C	0x006282b3	add x5,x5,x6	90: add t0, t0, t1

Address	Value (+0)	Value (+4)	Value (+8)	Value (+C)	Value (+10)	Value (+14)	Value (+18)	Value (+1C)
0x00006000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 13: Data segment before executing instructions

The screenshot shows the RARS IDE interface after execution. The 'Text Segment' table is identical to Figure 13. The 'Data Segment' table shows that the value at address 0x00006020 has been updated to 0x000079bd, which matches the 'answer' value from the test case.

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	46: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x00000313	addi x6,x0,0	47: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x00000393	addi x7,x0,0	48: addi t2, zero, 0 # t2 is the SW third read
	0x0000000C	0x00006437	lui x8,6	56: lui s0, 0x00006
	0x00000010	0x00040413	addi x8,x8,0	57: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	59: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	60: addi s1, s1, 0x020
	0x0000001C	0x00001e37	lui x28,1	66: lui t3, 0x00001
	0x00000020	0x234e0e13	addi x28,x28,0x0000234	67: addi t3, t3, 0x234
	0x00000024	0x01c42023	sw x28,0(x8)	68: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	71: lhu t0, 0(s0) # first read
	0x0000002C	0x12345e37	lui x28,0x00012345	74: lui t3, 0x12345
	0x00000030	0x678e0e13	addi x28,x28,0x00000678	75: addi t3, t3, 0x678
	0x00000034	0x01c42023	sw x28,0(x8)	76: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	79: lhu t1, 0(s0) # second read
	0x0000003C	0x11111e37	lui x28,0x00011111	82: lui t3, 0x11111
	0x00000040	0x111e0e13	addi x28,x28,0x00000111	83: addi t3, t3, 0x111
	0x00000044	0x01c42023	sw x28,0(x8)	84: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	87: lhu t2, 0(s0) # third read
	0x0000004C	0x006282b3	add x5,x5,x6	90: add t0, t0, t1

Address	Value (+0)	Value (+4)	Value (+8)	Value (+C)	Value (+10)	Value (+14)	Value (+18)	Value (+1C)
0x00006000	0x11111111	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x000079bd	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 14: Data segment after executing instructions

Result at LEDS address in 14 matches the answer in 12.

- Test Case 2

```

46 # TC2 : Maxed
47 .equ READ1_H,      0x00005 # switch first read (0x5555)
48 .equ READ1_L,      0x555
49
50 .equ READ2_H,      0x00005 # switch second read (0x5555)
51 .equ READ2_L,      0x555 # sign extended
52
53
54 .equ READ3_H,      0x00005 # switch third read (0x5555)
55 .equ READ3_L,      0x555
56 # answer= 0xFFFF

```

Figure 15: TC2 given input

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	55: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x0000313	addi x6,x0,0	56: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x0000393	addi x7,x0,0	57: addi t2, zero, 0 # t2 is the SW third read
	0x0000000c	0x00006437	lui x8,6	65: lui s0, 0x00006
	0x00000010	0x00049413	addi x8,x8,0	66: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	68: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	69: addi s1, s1, 0x020
	0x0000001c	0x00005e37	lui x28,5	75: lui t3, 0x00005
	0x00000020	0x555e0e13	addi x28,x28,0x00005555	76: addi t3, t3, 0x555
	0x00000024	0x01c42023	sw x28,0(x8)	77: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	80: lhu t0, 0(s0) # first read
	0x0000002c	0x00005e37	lui x28,5	83: lui t3, 0x00005
	0x00000030	0x555e0e13	addi x28,x28,0x00005555	84: addi t3, t3, 0x555
	0x00000034	0x01c42023	sw x28,0(x8)	85: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	88: lhu t1, 0(s0) # second read
	0x0000003c	0x00005e37	lui x28,5	91: lui t3, 0x00005
	0x00000040	0x555e0e13	addi x28,x28,0x00005555	92: addi t3, t3, 0x555
	0x00000044	0x01c42023	sw x28,0(x8)	93: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	96: lhu t2, 0(s0) # third read
	0x0000004c	0x006282b3	add x5,x5,x6	99: add t0, t0, t1

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00006000	0x00005555	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x0000ffff	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 16: Data segment after executing instructions

Result at LEDS address in 16 matches the answer in 15.

• Test Case 3

```

59 # TC3 : Overflow
60 .equ READ1_H,      0x00005 # switch first read (0x5555)
61 .equ READ1_L,      0x555
62
63 .equ READ2_H,      0x00005 # switch second read (0x5555)
64 .equ READ2_L,      0x555 # sign extended
65
66
67 .equ READ3_H,      0x00005 # switch third read (0x5557)
68 .equ READ3_L,      0x557
69 # answer= 0x0001

```

Figure 17: TC3 given input

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	66: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x0000313	addi x6,x0,0	67: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x0000393	addi x7,x0,0	68: addi t2, zero, 0 # t2 is the SW third read
	0x0000000c	0x00006437	lui x8,6	76: lui s0, 0x00006
	0x00000010	0x00049413	addi x8,x8,0	77: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	79: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	80: addi s1, s1, 0x020
	0x0000001c	0x00005e37	lui x28,5	86: lui t3, 0x00005
	0x00000020	0x555e0e13	addi x28,x28,0x00005555	87: addi t3, t3, 0x555
	0x00000024	0x01c42023	sw x28,0(x8)	88: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	91: lhu t0, 0(s0) # first read
	0x0000002c	0x00005e37	lui x28,5	94: lui t3, 0x00005
	0x00000030	0x555e0e13	addi x28,x28,0x00005555	95: addi t3, t3, 0x555
	0x00000034	0x01c42023	sw x28,0(x8)	96: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	99: lhu t1, 0(s0) # second read
	0x0000003c	0x00005e37	lui x28,5	102: lui t3, 0x00005
	0x00000040	0x557e0e13	addi x28,x28,0x0000557	103: addi t3, t3, 0x557
	0x00000044	0x01c42023	sw x28,0(x8)	104: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	107: lhu t2, 0(s0) # third read
	0x0000004c	0x006282b3	add x5,x5,x6	110: add t0, t0, t1

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00006000	0x00005557	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x00000001	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 18: Data segment after executing instructions

Result at LEDS address in 18 matches the answer in 17.

• Test Case 4

24

Figure 19: TC4 given input

[illegible]

Figure 20: Data segment after executing instructions

Result at LEDS address in 20 matches the answer in 19.

- Test Case 5

11

Figure 21: TC5 given input

[illegible]

Figure 22: Data segment after executing instructions

Result at LEDS address in 22 matches the answer in 21.

- Test Case 6

```

96
97 # TC6 : All highs into full
98 .equ READ1_H,      0x00008 # switch first read
99 .equ READ1_L,      0x000
100
101 .equ READ2_H,      0x00008 # switch second read
102 .equ READ2_L,      0x000
103
104 .equ READ3_H,      0x00008 # switch third read
105 .equ READ3_L,      0x000
106
107 # answer= 0x8000 (0x18000 as full word though)

```

Figure 23: TC6 given input

Text Segment									
Bkpt	Address	Code	Basic	Source					
	0x00000000	0x00000293	addi x5,x0,0	99:	addi t0, zero, 0	# t0 is the SW first read			
	0x00000004	0x00000313	addi x6,x0,0	100:	addi t1, zero, 0	# t1 is the SW second read			
	0x00000008	0x00000363	addi x7,t0,0	101:	addi t2, zero, 0	# t2 is the SW third read			
	0x0000000c	0x00006437	lui x8,6	109:	lui s0, 0x00006				
	0x00000010	0x00040413	addi x8,x8,0	110:	addi s0, s0, 0x000				
	0x00000014	0x000064b7	lui x9,6	112:	lui s1, 0x00006				
	0x00000018	0x02048493	addi x9,x9,0x00000020	113:	addi s1, s1, 0x020				
	0x0000001c	0x00008e37	lui x28,8	119:	lui t3, 0x00008				
	0x00000020	0x000e6e13	ori x28,x28,0	120:	ori t3, t3, 0x000				
	0x00000024	0x01c42023	sw x28,0(x8)	121:	sw t3, 0(s0)	# only 3 bytes are store...			
	0x00000028	0x00045283	lhu x5,0(x8)	124:	lhu t0, 0(s0)	# first read			
	0x0000002c	0x00008e37	lui x28,8	127:	lui t3, 0x00008				
	0x00000030	0x000e6e13	ori x28,x28,0	128:	ori t3, t3, 0x000				
	0x00000034	0x01c42023	sw x28,0(x8)	129:	sw t3, 0(s0)	# only 3 bytes are store...			
	0x00000038	0x00045283	lhu x6,0(x8)	132:	lhu t1, 0(s0)	# second read			
	0x0000003c	0x00008e37	lui x28,8	135:	lui t3, 0x00008				
	0x00000040	0x000e6e13	ori x28,x28,0	136:	ori t3, t3, 0x000				
	0x00000044	0x01c42023	sw x28,0(x8)	137:	sw t3, 0(s0)	# only 3 bytes are store...			
	0x00000048	0x00045283	lhu x7,0(x8)	140:	lhu t2, 0(s0)	# third read			
	0x0000004c	0x006282b3	add x5,x5,x6	143:	add t0, t0, t1				

Data Segment									
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)	
0x00006000	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	
0x00006020	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	

Figure 24: Data segment after executing instructions

Result at LEDS address in 24 matches the answer in 23.

Assembly Source Code

Listing 6: Question 1 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory three consecutive
4 #           times, summing the unsigned half-word reads and writing
5 #           to the LEDS addr.
6 #
7 # Notes :
8 #   - Assume caller/callee has no need to preserve reg values.
9 #   - KISS method for this assignment
10 #
11 #   - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 .equ SWITCHES_H, 0x11000 # upper 20 bits
19 .equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L, 0x000 # lower 12 bits
22
23 .equ LEDS_H, 0x11000 # upper 20 bits
24 .equ LEDS_L, 0x020 # lower 12 bits
25 .equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L, 0x020 # lower 12 bits
27
28 # ~~~~~ Test Cases ~~~~~
29 # TC1 : Word Clip
30 .equ READ1_H, 0x00001 # switch first read (0x1234)
31 .equ READ1_L, 0x234
32
33 .equ READ2_H, 0x12345 # switch second read (0x5678)
34 .equ READ2_L, 0x678
35
36 .equ READ3_H, 0x11111 # switch third read (0x1111)
37 .equ READ3_L, 0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 .equ READ1_H, 0x00005 # switch first read (0x5555)

```

```

42 #.equ READ1_L,      0x555
43
44 #.equ READ2_H,      0x00005 # switch second read (0x5555)
45 #.equ READ2_L,      0x555 # sign extended
46
47 #.equ READ3_H,      0x00005 # switch third read (0x5555)
48 #.equ READ3_L,      0x555
49 # answer= 0xFFFF
50
51 # TC3 : Overflow
52 #.equ READ1_H,      0x00005 # switch first read (0x5555)
53 #.equ READ1_L,      0x555
54
55 #.equ READ2_H,      0x00005 # switch second read (0x5555)
56 #.equ READ2_L,      0x555 # sign extended
57
58 #.equ READ3_H,      0x00005 # switch third read (0x5557)
59 #.equ READ3_L,      0x557
60 # answer= 0x0001
61
62 # TC4 : zero
63 #.equ READ1_H,      0x00000 # switch first read (0)
64 #.equ READ1_L,      0x0
65
66 #.equ READ2_H,      0x00000 # switch second read (0)
67 #.equ READ2_L,      0x0 # sign extended
68
69 #.equ READ3_H,      0x00000 # switch third read (0)
70 #.equ READ3_L,      0x0
71 # answer= 0x0000
72
73 # TC5 : MAXXXED
74 #.equ READ1_H,      0x0000F # switch first read (0xFFFF)
75 #.equ READ1_L,      -1
76
77 #.equ READ2_H,      0xFFFFF # switch second read (0xFFFF)
78 #.equ READ2_L,      -1
79
80 #.equ READ3_H,      0x00000 # switch third read (0)
81 #.equ READ3_L,      0
82 # answer= 0xFFFE
83
84 # TC6 : All highs into full
85 .equ READ1_H,      0x00008 # switch first read
86 .equ READ1_L,      0x000
87
88 .equ READ2_H,      0x00008 # switch second read
89 .equ READ2_L,      0x000
90
91 .equ READ3_H,      0x00008 # switch third read
92 .equ READ3_L,      0x000
93 # answer= 0x8000 (0x18000 as full word though)
94 # ~~~~~
95
96 main:
97
98 # ~~~~ register init ~~~~
99 addi t0, zero, 0 # t0 is the SW first read
100 addi t1, zero, 0 # t1 is the SW second read
101 addi t2, zero, 0 # t2 is the SW third read
102 # t3 is a dummy variable for testing
103
104 # s0 contains the SWITCH addr
105 # s1 contains the LEDS addr
106
107
108 # ~~~~ load addr into registers ~~~~
109 lui s0, SWITCHES_H
110 addi s0, s0, SWITCHES_L
111
112 lui s1, LEDS_H
113 addi s1, s1, LEDS_L
114
115
116 # ~~~~ read SWITCHES and sum ~~~~
117
118 # ~~ behind the scenes : switch mem changes value ~~
119 lui t3, READ1_H
120 ori t3, t3, READ1_L
121 sw t3, 0(s0) # only 3 bytes are stored (0x234)
122 # ~~~~~
123
124 lhu t0, 0(s0) # first read
125
126 # ~~ behind the scenes : switch mem changes value ~~
127 lui t3, READ2_H
128 ori t3, t3, READ2_L
129 sw t3, 0(s0) # only 3 bytes are stored (0x678)
130 # ~~~~~
131
132 lhu t1, 0(s0) # second read

```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)      # only 3 bytes are stored (0x111)
138 # ~~~~~
139
140 lhu t2, 0(s0)      # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10 # load system call number for exit()
153 li a0, 0  # Load the exit status code ( EXIT_SUCCESS )
154 ecall    # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

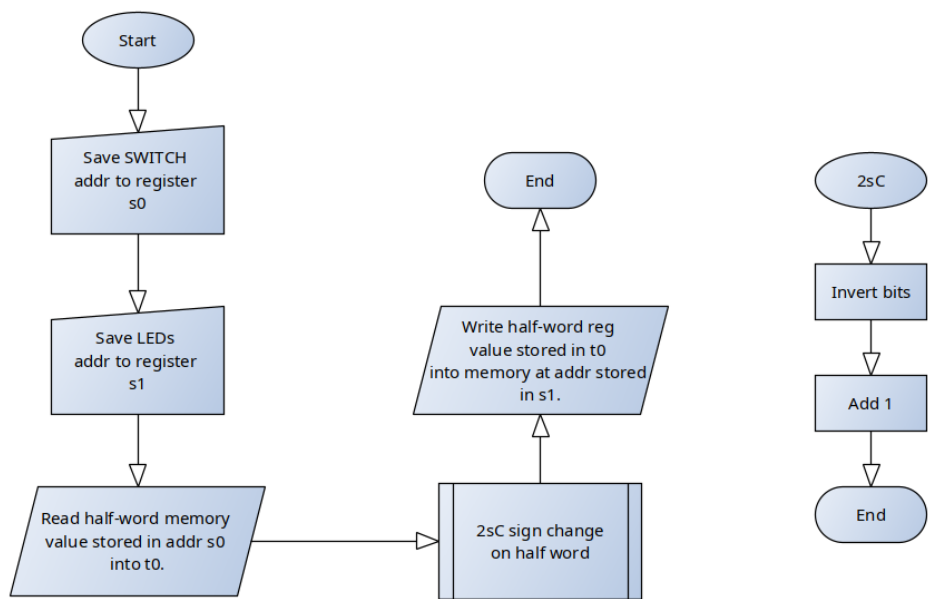


Figure 25: Flow Chart for Part 2

Verification Simulations

Test cases passed with definitions defined in code below.

```
16 # TC1 : zero
17 #A: .half 0
18 # answer= 0x0000
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000000
0x00006020	0x00000000

Figure 26: IO and Data Segment of TC1

```
21 # TC2 : positive
22 #A: .half 0x5
23 # answer= 0xFFFFB (-5)
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000005
0x00006020	0x0000ffffb

Figure 27: IO and Data Segment of TC2

25	# TC3 : negative		
26	#A: .half 0xFFFB	# -5	
27	# answer= 0x0005	# 5	
28			

Data Segment	
Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000005

Figure 28: IO and Data Segment of TC3

30	# TC4 : positive		
31	#A: .half 0x5678	# 22136	
32	# answer= 0xA988	# -22136	

Data Segment	
Address	Value (+0)
0x00006000	0x00005678
0x00006020	0x0000a988

Figure 29: IO and Data Segment of TC4

34	# TC5 : reverse		
35	#A: .half 0xA988		
36	# answer= 0x5678		
37			

Data Segment	
Address	Value (+0)
0x00006000	0x0000a988
0x00006020	0x00005678

Figure 30: IO and Data Segment of TC5

39	# TC6 : negative		
40	#A : .half 0xFFFF #(-1)		
41	# answer= 1		
42			

Data Segment	
Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000001

Figure 31: IO and Data Segment of TC6

43	# TC7 : negative 2		
44	#A : .half 0xFFFE #(-2)		
45	# answer= 2		
46			

Data Segment	
Address	Value (+0)
0x00006000	0x0000fffe
0x00006020	0x00000002

Figure 32: IO and Data Segment of TC7

48	# TC7 : negative 32 bit		
49	#A : .word 0x89ABCDEF	#(0xCDEF = -12817)	
50	# answer= 0x3211	# 12817	
51			

Data Segment	
Address	Value (+0)
0x00006000	0x89abcdef
0x00006020	0x00003211

Figure 33: IO and Data Segment of TC8

Assembly Code

Listing 7: Question 2 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory a unsigned half-word
4 #           changing the sign, then writing to the LEDS addr.
5 #
6 # Notes :
7 #     - Assume caller/callee has no need to preserve reg values.
8 #       KISS method for this assignment
9 #
10 #     - Split 32-bit addresses according to instr imm handling.
11
12 .data
13 # ~~~~~ Test Cases ~~~~~
14 # TC1 : zero
15 #A: .half 0
16 # answer= 0x0000
17
18 # TC2 : positive
19 #A: .half 0x5
20 # answer= 0xFFFB (-5)
21
22 # TC3 : negative
23 #A: .half 0xFFFB # -5
24 # answer= 0x0005 # 5
25
26 # TC4 : positive
27 #A: .half 0x5678 # 22136

```

```

28 # answer= 0xA988      # -22136
29
30 # TC5 : reverse
31 #A: .half 0xA988
32 # answer= 0x5678
33
34 # TC6 : negative
35 #A : .half 0xFFFF #(-1)
36 # answer= 1
37
38 # TC7 : negative 2
39 #A : .half 0xFFFE #(-2)
40 # answer= 2
41
42 # TC7 : negative 32 bit
43 #A : .word 0x89ABCDEF    #(0xCDEF = -12817)
44 # answer= 0x3211      # 12817
45
46 # ~~~~~
47
48
49 .global _start
50 .text
51 #.equ SWITCHES_H, 0x11000 # upper 20 bits
52 #.equ SWITCHES_L, 0x000 # lower 12 bits
53 #.equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
54 #.equ SWITCHES_L, 0x000 # lower 12 bits
55
56 #.equ LEDS_H, 0x11000 # upper 20 bits
57 #.equ LEDS_L, 0x020 # lower 12 bits
58 #.equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
59 #.equ LEDS_L, 0x020 # lower 12 bits
60
61
62
63 _start:
64
65 # ~~~~ register init ~~~~
66 addi t0, zero, 0 # t0 is the SW first read
67 # t3 is a dummy variable for testing
68
69 # s0 contains the SWITCH addr
70 # s1 contains the LEDS addr
71
72 # ~~~~ load addr into registers ~~~~
73 lui s0, SWITCHES_H
74 addi s0, s0, SWITCHES_L
75
76 lui s1, LEDS_H
77 addi s1, s1, LEDS_L
78
79
80 # ~~~~ read SWITCHES and sum ~~~~
81
82 # ~~ behind the scenes : switch value ~~
83 lw t3, A
84 sw t3, 0(s0)
85 # ~~~~~
86
87 lhu t0, 0(s0) # first read
88
89
90 # ~~~~ invert the sign (2sC) ~~~~
91 not t0, t0
92 addi t0, t0, 1
93
94 # ~~~~ Write the unsigned half word to LEDS mem addr
95 sh t0, 0(s1)
96
97
98 # ~~~~ return with exit status ~~~~
99 li a7, 10 # load system call number for exit()
100 li a0, 0 # Load the exit status code ( EXIT_SUCCESS )
101 ecall # Make the system call

```

0.2.2 SW2 - Conditional Statements

- Assignment:**
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.
1. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input is greater than or equal to 32,768, the value is divided by 4. You can ignore any remainder. If the value is less than 32,768, the value is multiplied by 2. The result should be written to the 7 SEGMENT (address 0x11000040).
 2. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input value is a multiple of 4, all of the bits should be inverted, otherwise, if the input value is odd, add 4095 and divide the result by 2, otherwise subtract 1 from the value. The result should be written to the 7 SEGMENT (address 0x11000040).

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 34: Questions to address

Part 1

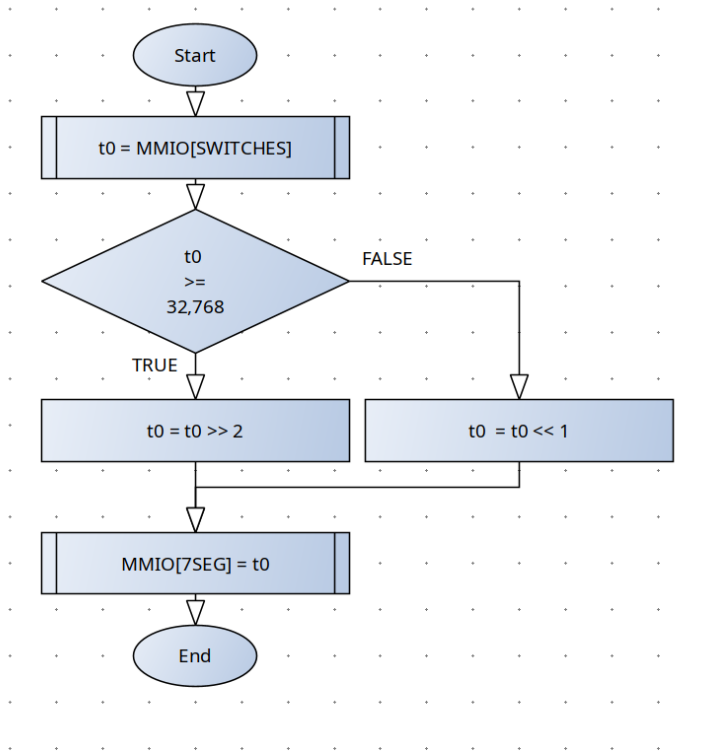


Figure 35: Flow chart for part 1

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Lowest unsigned val	0	0	Passed
2	Below boundary	0xFFF	0x1FFE	Passed
3	1 before boundary	0x7FFF	0xFFFE	Passed
4	On boundary	0x8000	0x2000	Passed
5	1 above boundary	0x8001	0x2000	Passed
6	Above boundary	0x12345678	0x048D159E	Passed
7	32-bit unsigned ceiling	0xFFFFFFFF	0x3FFFFFFF	Passed

Table 2: Test cases for part 1

Listing 8: sw2_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Conditionals, branching, jumping
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #     constant CONST
8 #
9 # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero ~~
17 #TC:      .word 0
18 # answer = 0 (verified)
19
20 # ~~ TC2 : Under ~~
21 #TC:      .word 0xFFFF
22 # answer = 0x1FFE (verified)
23
24 # ~~ TC3 : One before ~~
25 #TC:      .word 0x07FFF
26 # answer = 0xFFFE (verified)
27
28 # ~~ TC4 : On boundary ~~
29 #TC:      .word 0x08000
30 # answer = 0x2000 (verified)
31
32 # ~~ TC5 : Above boundary ~~
33 #TC:      .word 0x08001
34 # answer = 0x2000 (verified)
35
36 # ~~ TC6 : Above boundary 2 ~~
37 #TC:      .word 0x12345678
38 # answer = 0x048D159E (verified)
39
40 # ~~ TC7 : 32-bit unsigned ceiling ~~
41 #TC:      .word 0xFFFFFFFF
42 # answer = 0x3FFFFFFF (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49 .equ CONST, 32768
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li t1, MMIO_SWITCHES    # t1 is the addr for MMIO SWITCHES
54 li t2, CONST            # t2 is the constant 32768
55 li t3, MMIO_SEVSEG      # t3 is the addr for MMIO 7SEG
56                          # t4 is a test case variable
57
58 # ~~~~ TC injection ~~~~
59 lw t4, TC
60 sw t4, 0(t1)
61
62 # ~~~~ read SWITCHES ~~~~
63 lw t0, 0(t1)            # load SW value into t0
64
65 # ~~~~ unsigned branch ~~~~
66 bgeu t0, t2, Div4
67
68 # ~~ else (false) case ~~
69 slli t0, t0, 1          # multiply by 2
70 j End
71
72 # ~~ if (true) case ~~
73 Div4:
74 srli t0, t0, 2          # logical shift for unsigned
75
76 # ~~~~ write to 7SEG ~~~~
77 End:
78 sw t0, 0(t3)
79
80 # ~~~~ syscall exit status ~~~~
81 li a7, 10               # load syscall num for exit()
82 li a0, 0                # EXIT_SUCCESS code
83 ecall                  # execute sys call
84
85

```

Figure 36: Assembly code for SW2 part 1

Part 2

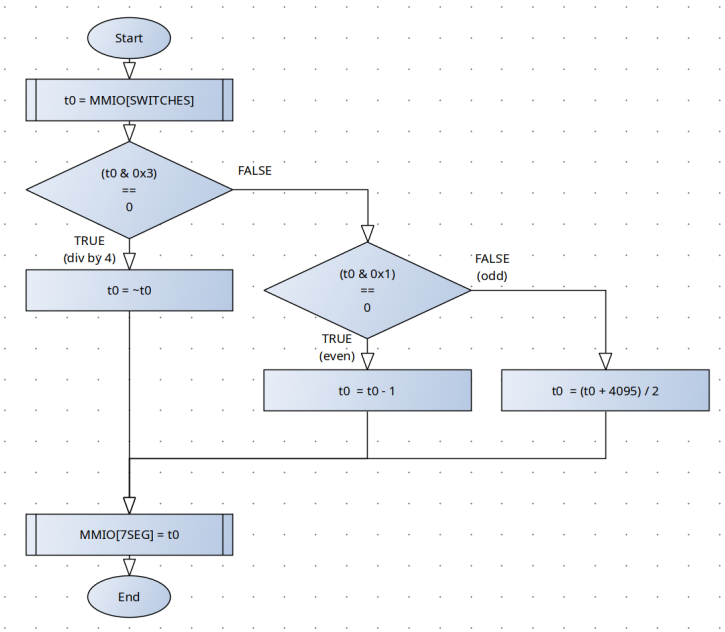


Figure 37: Flow chart for part 2

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Zero is div by 4	0	0xFFFFFFFF	Passed
2	Two is even	0x2	0x1	Passed
3	One is odd	0x1	0x0800	Passed
4	32-bit unsinged ceiling	0xFFFFFFFF	0x07FF	Passed
5	Div by 4 near ceiling	0xFFFFFFFFC	0x3	Passed
6	Odd near ceiling	0xFFFFFFFDD	0x07FE	Passed
7	Even near ceiling	0xFFFFFFFEE	0xFFFFFFFDD	Passed

Table 3: Test cases for part 1

Listing 9: sw2_p2.asm

```

1  # Author : WhisperingRock
2  #
3  # Purpose : Conditionals, branching, jumping
4  # Input :
5  #     SWITCHES MMIO address to 32-bit unsigned value
6  #     7SEG MMIO address to 8-digits, each of 4-bits
7  #     constant CONST
8  #
9  # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero (Div4) ~~
17 #TC:      .word 0
18 # answer = 0xFFFF_FFFF (verified)
19
20 # ~~ TC2 : Two (Div2 = even) ~~
21 #TC:      .word 2
22 # answer = 0x1 (verified)
23
24 # ~~ TC3 : 1 (odd) ~~
25 #TC:      .word 0x1
26 # answer = 0x0800 (verified)
27
28 # ~~ TC4 : 32-bit unsigned ceiling ~~
29 #TC:      .word 0xFFFFFFFF
30 # answer = 0x0000_07FF | 2047 (verified)
31
32 # ~~ TC5 : unsigned ceiling Div4 ~~
33 #TC:      .word 0xFFFF_FFC
34 # answer = 0x03 (verified)
35
36 # ~~ TC6 : unsigned ceiling odd ~~
37 #TC:      .word 0xFFFF_FFD
38 # answer = 0x07FE (verified)
39
40 # ~~ TC6 : unsigned ceiling even ~~
41 #TC:      .word 0xFFFF_FFE
42 # answer = 0xFFFF_FFD (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
54 li s2, MMIO_SEVSEG   # s2 is the addr for MMIO 7SEG
55                       # t0 is the working variable for SWITCHES value
56                       # t1 is a logic case variable
57 li t2, 4095          # t2 is reserved for const
58
59 # ~~~~ TC injection ~~~~
60 lw t0, TC
61 sw t0, 0(s1)
62
63 # ~~~~ read SWITCHES ~~~~
64 lw t0, 0(s1) # load SW value into t0
65
66 # ~~~~ branch : div by 4 ~~~~
67 andi t1, t0, 0x3
68 beqz t1, Div4
69
70 # ~~~~ branch : div by 2 ~~~~
71 andi t1, t0, 0x1
72 beqz t1, Div2
73
74 # ~~ else case ~~
75 add t0, t0, t2 # add const 4095
76 srli t0, t0, 1 # then scale
77 j End
78
79 Div4:
80 not t0, t0 # invert bits
81 j End
82
83 Div2:
84 addi t0, t0, -1
85
86 End: # ~~~~ write to 7SEG ~~~~
87 sw t0, 0(s2)
88
89
90
91 # ~~~~ syscall exit status ~~~~
92 li a7, 10 # load syscall num for exit()23
93 li a0, 0 # EXIT_SUCCESS code
94 ecalls # execute sys call
95
96

```

0.3 Appendix